TP10 WINDOWS / MLOPS

API de détection de fraude

Comparaison Flask et FastAPI,
mesures de charge et sécurisation

0,8899

Probabilité de fraude
pour la transaction Benali

Python 3.11 sur Windows / scikit-learn / Docker Desktop

VUE D’ENSEMBLE

Les 17 étapes du TP10

Le même modèle de classification sert aux prototypes, aux tests de charge et aux essais de sécurité.

PHASE	ÉTAPES	TRAVAIL RÉALISÉ
A / Comparaison	1 à 2	Préparer le modèle TP3 et comparer Flask et FastAPI.
B / Service REST	3 à 7	Versionner les routes, valider, tester et mesurer la charge.
C / Sécurité	8 à 16	Monter deux variantes FastAPI et vérifier huit protections.
Clôture	17	Nettoyer le laboratoire et conserver les résultats et l’image sécurisée.

Fil conducteur / vérifier que le score de référence reste à 0,8899.

Le modèle TP3 et la transaction Benali

Le dossier tp10-api contient une copie du modèle entraîné et un client HTTP en ligne de commande.

REQUÊTE JSON

```
{
  "montant": 1800,
  "ancienneté": 30,
  "freq_24h": 4,
  "nuit": 1
}
```

PRÉDICTION DE RÉFÉRENCE

0,8899

fraude = true
Le seuil de décision est p > 0,5.

Le modèle TP3, sa copie et le fichier du conteneur ont le même SHA-256.

A / ÉTAPE 2

Flask et FastAPI sur une entrée valide

Même corps JSON, même modèle et même route POST /predict.

FLASK / PORT 5001

200 OK

```
{"fraude":true,"probabilite":0.8899}
```

Lit le JSON avec request.get_json() et sélectionne les quatre variables.

FASTAPI / PORT 8000

200 OK

```
{"fraude":true,"probabilite":0.8899}
```

Valide les types avec Pydantic, puis transforme le résultat en variables du modèle.

Sur ce cas valide, le choix du framework ne change pas la prédiction.

A / ÉTAPE 2

Flask et FastAPI sur une entrée invalide

Un seul champ change dans la requête. La valeur de montant devient « abc ».

FLASK / TRAITEMENT DIRECT DU JSON

500

```
500 INTERNAL SERVER ERROR
<h1>Internal Server Error</h1>
```

Le texte arrive jusqu'au calcul. La conversion échoue et Flask renvoie une page HTML.

FASTAPI / VALIDATION PYDANTIC

422

```
422 Unprocessable Entity
"type":"float_parsing"
"loc":["body","montant"]
```

Pydantic rejette le texte avant la prédiction. La réponse JSON identifie le champ à corriger.

Dans ces prototypes, FastAPI donne au client une erreur qu'il peut corriger.

Sorties réelles de POST /predict, extraits

Les routes de l’API REST

Le modèle se charge une fois au démarrage. Les routes métier sont versionnées sous /v1.

REQUÊTE	RÉSULTAT	RÔLE OU CAUSE
GET /health	200	État du service et présence du modèle.
GET /v1/model-info	200	Version 1.0.0 et liste des variables.
POST /v1/predict	200	Score, décision, version et ID de requête.
POST /v1/predict invalide	422	Le corps ne respecte pas le schéma.
GET /v1/predict	405	La route existe et attend POST.
POST /predict	404	Le chemin du prototype est absent de cette API.

Le code HTTP indique au client s’il doit corriger le chemin, la méthode ou les données.

Contrat d’entrée et de sortie

Le schéma impose des types, des bornes et quatre champs obligatoires.

ENTRÉE / TRANSACTION

CHAMP	TYPE	BORNE
montant	Flottant	0 < montant ≤ 1 000 000 MAD
anciennete	Entier	0 à 600 mois
freq_24h	Entier	0 à 500 opérations
nuit	Entier	0 ou 1

SORTIE / PREDICTION

```
{
  "fraude": true,
  "probabilite": 0.8899,
  "version_modele": "1.0.0",
  "id_requete": "a1e67cee"
}
```

extra="forbid" rejette aussi tout champ supplémentaire.

Le journal associe l’ID, la version, le score et la durée à chaque prédiction.

Swagger et tests automatisés

Swagger permet un essai manuel. Pytest vérifie le comportement attendu à chaque exécution.

CINQ TESTS RÉUSSIS

CAS	HTTP
Service prêt	200
Benali à 0,8899	200
Montant négatif	422
Champ manquant	422
Champ inconnu	422

RÉPONSE DANS SWAGGER



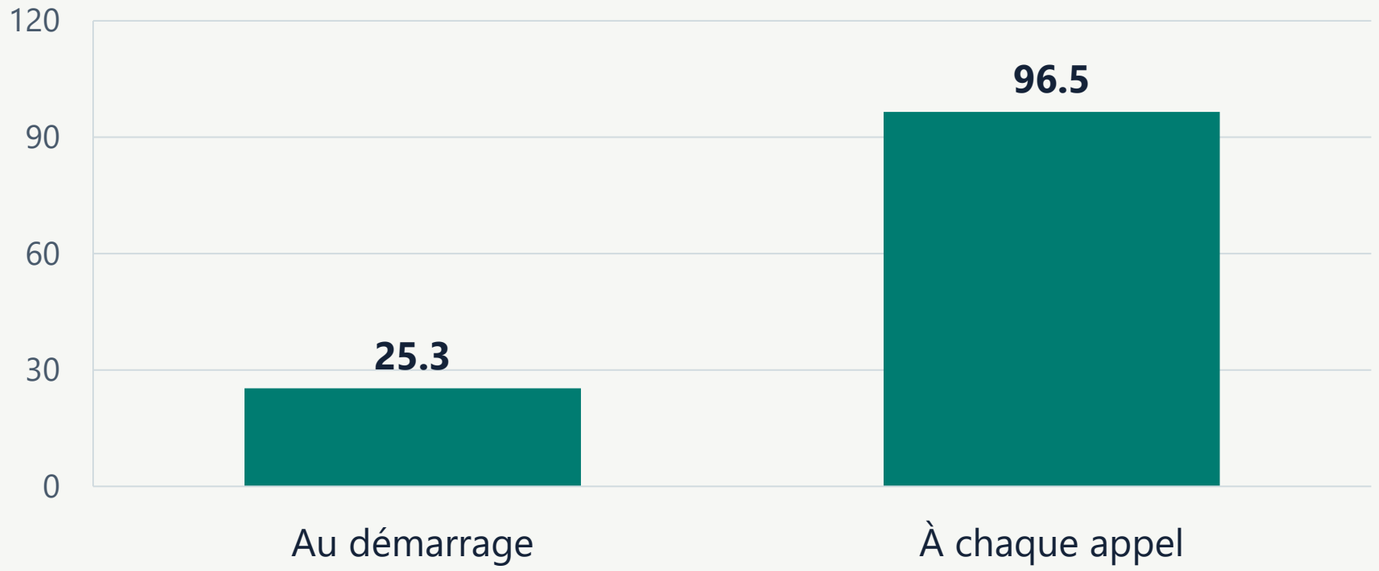
Essai Swagger / montant = 0, réponse 422

Capture Swagger, test_api.py et sortie pytest

Le coût du rechargement du modèle

200 requêtes par essai avec un seul client. La prédiction reste à 0,8899.

LATENCE MÉDIANE EN MS



MESURES COMPLÉMENTAIRES

CHARGEMENT	REQ/S	P95
Au démarrage	31	48,2 ms
À chaque appel	10	123,4 ms

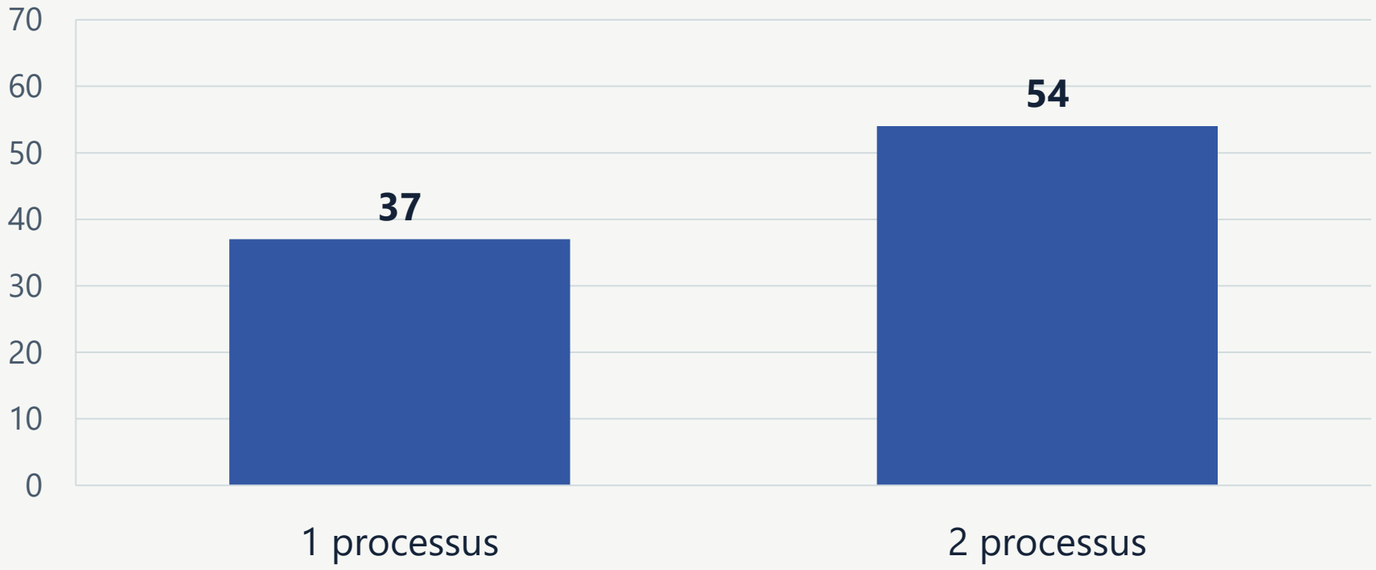
Le p95 indique le délai sous lequel 95 % des requêtes ont été traitées.

25,3 ms contre 96,5 ms. Le rechargement multiplie la médiane par 3,81.

Un ou deux processus Uvicorn

400 requêtes par essai avec 20 clients concurrents. Le modèle se charge au démarrage.

DÉBIT EN REQUÊTES PAR SECONDE



LATENCE OBSERVÉE

PROCESSUS	MÉDIANE	P95
1	531,7 ms	629,1 ms
2	358,6 ms	576,5 ms

Les processus traitent des requêtes en parallèle. Chacun conserve son propre état en mémoire.

Le débit passe de 37 à 54 req/s dans cet essai.

Deux variantes FastAPI dans le laboratoire

Même modèle et même route POST /v1/predict. Les différences viennent des protections activées.

ÉLÉMENT	BROUILLON	VERSION SÉCURISÉE
Serveur local	HTTP sur le port 8001	HTTPS sur le port 8443
En-tête X-API-Key	Clé de démonstration admin	Clé aléatoire lue dans API_KEY
Requête de prédiction	Corps JSON libre	Schéma Pydantic et limite de débit
Journaux	Clé et corps de requête	ID, version, score et durée

trustme crée les certificats locaux. Le client vérifie le certificat du serveur.

Les deux variantes retournent 0,8899 pour Benali avec leur clé respective.

Ce que le relais voit en HTTP et HTTPS

Le client passe par le même relais d'observation et obtient HTTP 200 dans les deux essais.

HTTP / TRAFIC EN CLAIR

```
POST /v1/predict HTTP/1.1
X-API-Key: admin
Content-Type: application/json

{"montant": 1800, "ancienete": 30,
 "freq_24h": 4, "nuit": 1}
```

Le relais peut lire la clé et les quatre valeurs de la transaction.

HTTPS / TRAFIC TLS

```
--- client -> serveur : 517 octets ---
16 03 01 02 00 01 00 01 fc 03 03 …

--- client -> serveur : 286 octets ---
17 03 03 01 19 31 4b 8c 9a f1 52 …
```

Le relais voit les octets TLS. La clé et le JSON sont protégés pendant le transport.

Une clé aléatoire protège contre la devinette. TLS protège la clé pendant le transport.

Attaque par dictionnaire

Le même script essaie une liste de clés sur la route de prédiction des deux variantes.

BROUILLON

11e essai

```
essai 10   cle='cle'     -> 401
essai 11   cle='admin'   -> 200
```

La clé connue permet une prédiction. Le script s'arrête dès le premier succès.

VERSION SÉCURISÉE

14 refus

```
essai 11   cle='admin'   -> 401
essai 14   cle='cle-tp-2026' -> 401
```

La variante sécurisée utilise secrets.compare_digest avec la clé générée localement.

Le quota s'applique après authentification. Une passerelle peut limiter les essais de clés invalides.

Clé d’API et images Docker

Deux images ont été construites localement. Leur contenu a été contrôlé avant le lancement.

CONTRÔLE	FRAUDE-API:MAUVAIS	FRAUDE-API:SECURISEE
Origine de la clé	ENV dans Dockerfile.mauvais	API_KEY fournie au lancement
docker history	La clé de démonstration apparaît	Aucune ligne API_KEY
printenv API_KEY dans une image lancée seule	La valeur de démonstration apparaît	Aucune valeur. Code de sortie 1.

```
docker run … --env-file .env fraude-api:securisee
```

En production, un coffre à secrets peut fournir la clé au démarrage.

.gitignore écarte .env de Git. .dockerignore l’écarte du contexte de construction.

Six entrées piégées et leurs réponses

Le brouillon accepte trois entrées et renvoie trois erreurs 500. Le schéma sécurisé rejette les six.

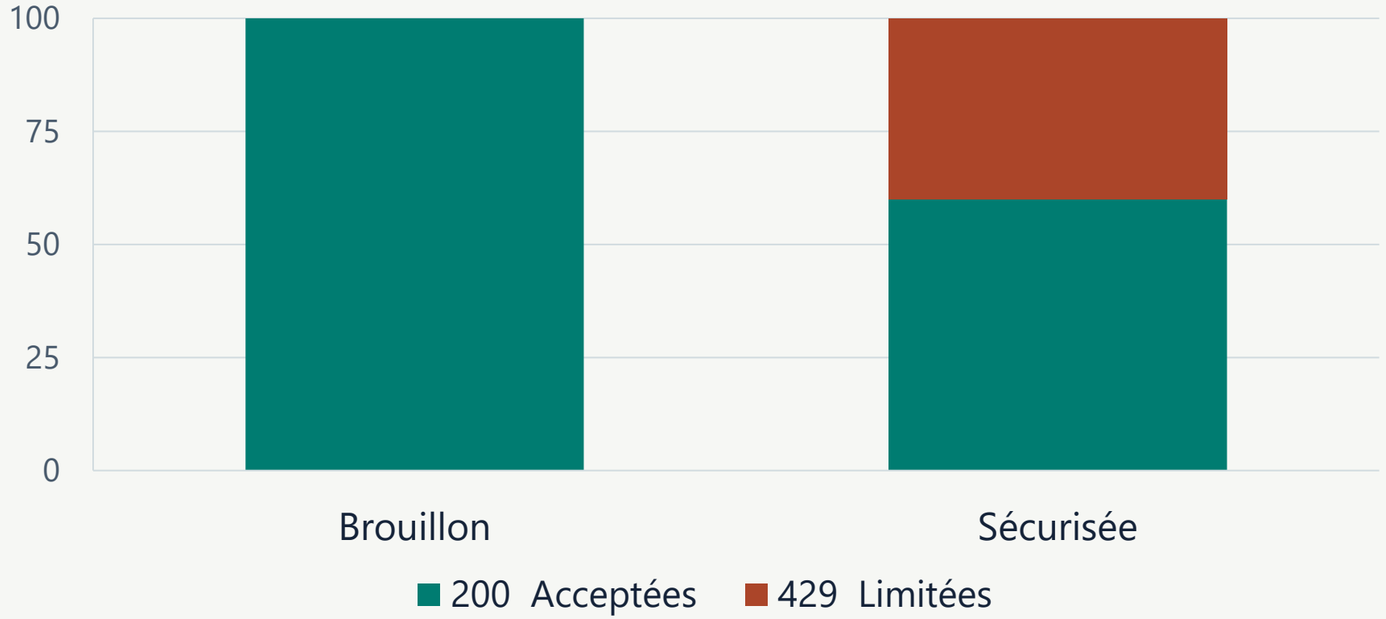
ENTRÉE TESTÉE	BROUILLON	VERSION SÉCURISÉE
montant = -5000	200 / prédiction	422 / montant > 0
nuit = 7	200 / prédiction	422 / nuit ∈ {0, 1}
montant = 1e300	500 / erreur	422 / montant ≤ 1 000 000
champ role ajouté	200 / champ ignoré	422 / champ supplémentaire
montant = "abc"	500 / erreur	422 / conversion en nombre
champ nuit absent	500 / erreur	422 / champ obligatoire

Un 200 sur une entrée impossible peut donner l'impression qu'une décision est valide.

Limite de débit sur 100 requêtes

Les appels sont authentifiés. La fenêtre de comptage est vide au début de l'essai sécurisé.

RÉPARTITION DES CODES HTTP



60 REQUÊTES PAR MINUTE

Le compteur utilise une fenêtre glissante par clé et par processus.

Deux workers peuvent accepter plus de 60 appels au total pour la même clé.

Brouillon : 100 réponses 200. Version sécurisée : 60 réponses 200 et 40 réponses 429.

Informations exposées par les erreurs

La même valeur « abc » révèle des informations différentes selon le traitement.

CONFIGURATION	CODE	TAILLE	CONTENU REÇU PAR LE CLIENT
Brouillon, debug=True	500	5 588 car.	Trace, chemins, ligne de code et bibliothèques
Brouillon, debug=False	500	21 car.	Internal Server Error
Version sécurisée	422	151 car.	Champ montant et erreur de type en JSON

L’essai a été rejoué après le passage du brouillon à debug=False.

La validation répond au client avant que la valeur invalide atteigne le modèle.

etape-14-erreurs.txt, tailles de réponse observées

Écriture dans le dossier du modèle

Chaque conteneur reçoit les mêmes commandes whoami et touch.

CONTENEUR MAUVAIS / PORT 8081

root

```
$ whoami
root
$ touch models/modele_pirate.pkl
$ ls models
model.pkl  modele_pirate.pkl
```

Le fichier de test est créé.

CONTENEUR SÉCURISÉ / PORT 8082

app

```
$ whoami
app
$ touch models/modele_pirate.pkl
Permission denied
$ ls models
model.pkl
```

L'écriture est bloquée.

Dans le conteneur sécurisé, Benali renvoie HTTP 200 et 0,8899 avec la clé.

Journaux de prédiction et d’accès

Les journaux du service sécurisé permettent de relier une prédiction à sa version et de repérer les refus.

BROUILLON / DONNÉES EXPOSÉES

```
cle=admin
 donnees={'montant': 1800,
 'anciennete': 30, 'freq_24h': 4,
 'nuit': 7}
```

VERSION SÉCURISÉE / MÉTADONNÉES

```
prediction id=b9288933
version=1.0.0  proba=0.8899
duree_ms=37.9
```

SIGNAUX DANS LE JOURNAL D’ACCÈS

401	422	429
Clé absente ou invalide	Données rejetées	Quota atteint

Le journal de prédiction conserve les métadonnées. Les codes HTTP tracent les refus.

Nettoyage et contrôle final

Les conteneurs de test sont supprimés. Le modèle et l’image sécurisée restent disponibles pour la suite.

ACTION	RÉSULTAT VÉRIFIÉ
Arrêt des serveurs et suppression des conteneurs	Le laboratoire n’écoute plus sur ses ports de test.
Suppression de fraude-api:mauvais	L’image de démonstration a été retirée.
Conservation de fraude-api:securisee	L’image locale reste disponible pour les TP11 et TP12.
Relance de l’image sécurisée avant publication	Benali à 0,8899, refus sans clé et écriture bloquée pour app.

Les fichiers .env, certs et .venv restent locaux.

Résultats à retenir

Les tests de contrat, les mesures de charge et les essais de sécurité répondent à des questions différentes.

QUESTION	RÉSULTAT DU TP
Le modèle donne-t-il le bon score ?	0,8899 pour Benali dans les prototypes et le conteneur sécurisé.
Le client sait-il corriger sa requête ?	FastAPI renvoie 422 et indique le champ. Le prototype Flask renvoie 500.
Où gagne-t-on du temps ?	Chargement unique : 25,3 ms de médiane. Deux processus : 54 req/s dans cet essai.
Où s’appliquent les protections ?	Au transport, dans le code de l’API et dans le conteneur.

Un compteur par processus et un certificat local décrivent le périmètre de ce laboratoire.

DOSSIER DE REMISE

Livrables et preuves

Les fichiers du TP10 sont disponibles sur la branche main de yPMDD/tp-mlops.

LIVRABLE	CONTENU
tp10-api	Code, modèle, cinq tests et fichiers Docker.
bilan-tp10.md	Quatre mesures de charge et huit comparaisons de sécurité.
compte-rendu-tp10.md	Résultats, preuves et réponses aux neuf questions.
captures/	Flask et FastAPI, relais HTTP/HTTPS, dictionnaire, privilèges Docker et traces complémentaires.

Sujet TP10 Windows et dossier de preuve

github.com/yPMDD/tp-mlops